REVA UNIVERSITY
Bengaluru, India
REVA Academy for Corporate Excellence (RACE)

Name: Hari Krishnan

SRN: AI09

Date: 03-May-2025

Module Name: Machine Learning

Answer Script

Answers	
Section:	
Q1.	a) The following table shows information about patients, including their BMI, Exercise Level, and whether they have a certain disease. Using KNN algorithm with an appropriate value of k, determine the predicted result for the 6th patient. Clearly show the steps involved, including distance calculations and final classification with justification. b) Describe and analyse the steps involved in training a Linear Regression model. Include the mathematical formulation, optimization process, and how model performance is evaluated using appropriate metrics. c) Analyze the role of the sigmoid activation function in Logistic Regression and explain how it helps in mapping predictions to probabilities.
Ans:	a) **K-Nearest Neighbors (KNN)** algorithm: The question provides data for five patients with information on their BMI, Exercise Level, and whether they have a disease. The goal is to predict whether the 6th patient has the disease using the **K-Nearest Neighbors (KNN)** algorithm. I will use **Euclidean distance** to measure the closeness and choose an appropriate value for **k**. Step 1: **Representing Data as Feature Vectors** Step 2: **Calculating Euclidean Distance** Step 3: **Choosing the Value of k** Step 4: **Identifying the 3 Nearest Neighbors** Step 5: **Majority Voting** **Step 1:** Representing Data as Feature Vectors Each patient is represented as a vector of two features: BMI and Exercise Level.

S. No	BMI	Exercise Level	Disease	Vector
1	8	2	Yes	[8, 2]
2	5	7	No	[5, 7]
3	6	6	No	[6, 6]
4	7	3	Yes	[7, 3]
5	4	8	No	[4, 8]
6	6	4	?	[6, 4]

Step 2: Calculating Euclidean Distance

The formula for Euclidean distance is:

Distance = $\sqrt{\{(x_2 - x_1)^2 + (y_2 - y_1)^2\}}$

I have calculated the distance of Patient 6 from each of the other patients:

To Patient 1 (Yes): Distance = √((8 - 6)² + (2 - 4)²) = √(4 + 4) = √8 approx 2.83

To Patient 2 (No): Distance = √((5 - 6)² + (7 - 4)²) = √(1 + 9) = √10 approx 3.16

To Patient 3 (No): Distance = √((6 - 6)² + (6 - 4)²) = √(0 + 4) = √4 = 2.00

To Patient 4 (Yes): Distance = √((7 - 6)² + (3 - 4)²) = √(1 + 1) = √2 approx 1.41

To Patient 5 (No): Distance = √((4 - 6)² + (8 - 4)²) = √(4 + 16) = √20 approx 4.47

Step 3: Choosing the Value of k

We need to choose a small odd number for k. I will choose **k = 3**.

Step 4: Identifying the 3 Nearest Neighbors

Based on the distances calculated:

Patient 4 (Yes) → 1.41

Patient 3 (No) → 2.00

Patient 1 (Yes) → 2.83

So, the 3 nearest neighbors are:

Patient 4 (Yes)

Patient 3 (No)

Patient 1 (Yes)

Step 5: Majority Voting

Among the 3 neighbors:

Yes = 2 votes (Patient 4 and 1)

No = 1 vote (Patient 3)

So, the majority is Yes.

Conclusion:

The 6th patient is predicted to have the disease (Yes) using the KNN algorithm with k = 3.

This is because the majority class among the three closest patients is "Yes".

b) Training a Linear Regression Model:

Linear Regression is a supervised learning algorithm used for predicting a continuous dependent variable (target) based on one or more independent variables (features).

The goal is to find the best-fitting straight line through the data points.

Training a linear regression model involves:

- Defining a linear equation,
- Minimizing the MSE cost function using gradient descent,
- Evaluating performance using metrics like MSE, MAE, RMSE, and R^2.

This process ensures that the model learns the best-fitting line for predicting outcomes from input features.

1. Mathematical Formulation

For a dataset with one feature, we can use this formula for the linear regression model:

$\hat{y} = w_1 x + w_0$

$\hat{y}$ is the predicted value,

x is the input feature,

w_1 is the weight (slope),

w_0 is the bias or intercept.

For multiple features:

$\hat{y} = w_0 + w_1x_1 + w_2x_2 + ... + w_nx_n$

(or)

$\hat{y} = Xw$

X is the feature matrix,

w is the weight vector.

2. Optimization – Cost Function

To train the model, we need to minimize the error between the predicted and actual values.

The commonly used error function is Mean Squared Error (MSE):

$MSE = (1/n) * \Sigma(y_i - \hat{y}_i)^2$

y_i is the actual value,

$\hat{y}_i$ is the predicted value,

n is the number of samples.

3. Gradient Descent (Optimization Algorithm)

To minimize MSE, we use Gradient Descent, which iteratively updates the weights:

$w = w - \alpha * \nabla(MSE)$

- α is the learning rate,
- $\nabla(MSE)$ is the gradient (partial derivatives) of the MSE with respect to each weight.

This process continues until the loss is minimized or the model converges.

4. Model Evaluation – Performance Metrics

After training, we evaluate the model's performance using appropriate metrics:

Mean Squared Error (MSE):

$MSE = (1/n) * \Sigma(y_i - \hat{y}_i)^2$

Measures the average squared difference between actual and predicted values.

Root Mean Squared Error (RMSE):

$RMSE = \sqrt{MSE}$

This gives the error in the same unit as the target variable.

Mean Absolute Error (MAE):

MAE = $(1/n) * \Sigma|y_i - \hat{y}_i|$

Measures the average absolute difference — less sensitive to outliers than MSE.

R^2 Score (Coefficient of Determination):

$R^2 = 1 - (SS\_res / SS\_tot)$

- SS_res is the residual sum of squares: $\Sigma(y_i - \hat{y}_i)^2$
- SS_tot is the total sum of squares: $\Sigma(y_i - \bar{y})^2$

R^2 indicates how well the model explains the variability in the data. $R^2 = 1$ means perfect prediction, and $R^2 = 0$ means no predictive power.

c) Role of the Sigmoid Activation Function in Logistic Regression

The sigmoid function is **crucial in logistic regression** because it enables the model to output valid probabilities, supports binary classification through thresholding and allows smooth optimization using gradient-based methods. Without the sigmoid function, logistic regression wouldn't be able to function as a probability-based classifier.

In **Logistic Regression**, the primary goal is to **predict the probability** that a given input belongs to a particular class — usually a binary outcome like Yes/No, 0/1, or True/False.
the raw output of a linear model:

$z = w_0 + w_1x_1 + w_2x_2 + ... + w_nx_n$

can range from $-\infty$ **to** $+\infty$, and probabilities must lie strictly between **0 and 1**.

The Role of the Sigmoid Function

To convert the linear output z into a **probability**, we apply the **sigmoid activation function**, defined as:

$\sigma(z) = 1 / (1 + e^{(-z)})$

This function "squashes" any real-valued number into the (0, 1) interval.

	Importance of Logistic Regression: **Probability Mapping:** The sigmoid function transforms the raw prediction z into a value between 0 and 1. This value represents the **probability** of the input belonging to the positive class (class = 1). **Decision Thresholding:** After computing the probability, we can choose a threshold (commonly 0.5) to classify the result: • If $\sigma(z) \geq 0.5$, predict class 1 • If $\sigma(z) < 0.5$, predict class 0 **Smooth Gradient for Training:** The sigmoid function is **differentiable**, which is essential for optimizing the model using **gradient descent**. It allows us to calculate how much to adjust the weights during training. **Visual Intuition** The sigmoid curve is S-shaped. As the input z increases positively, the output approaches 1. As z becomes more negative, the output approaches 0. Around z = 0, the curve is steep, which helps the model respond sensitively to small changes in input features.
Q2.	You have developed a classification model to predict whether a customer will churn (leave) or not churn (stay). In your dataset, only 12% of customers actually churn, making it highly imbalanced. After testing the model, the following confusion matrix is obtained. i. Calculate Accuracy, Precision, Recall, and F1-Score. ii. Is Accuracy the right metric to evaluate the model in this case? Justify your answer. iii. If your goal is to minimize customer loss by launching a retention campaign, should you prioritize Precision or Recall? Justify.
Ans:	a) i) Calculate Accuracy, Precision, Recall, and F1-Score TP (True Positive) = 30 FN (False Negative) = 90 FP (False Positive) = 40 TN (True Negative) = 840

1. **Accuracy** = (TP + TN) / (TP + TN + FP + FN)
= (30 + 840) / (30 + 840 + 40 + 90)
= 870 / 1000
= 0.87 or 87%
2. **Precision** = TP / (TP + FP)
= 30 / (30 + 40)
= 30 / 70
approx 0.4286 or 42.86%
3. **Recall** = TP / (TP + FN)
= 30 / (30 + 90)
= 30 / 120
= 0.25 or 25%
4. **F1-Score** = 2 × (Precision × Recall) / (Precision + Recall)
= 2 × (0.4286 × 0.25) / (0.4286 + 0.25)
= 2 × 0.10715 / 0.6786
approx 0.3156 or 31.56%

ii)

No, **accuracy is not the right metric** in this case because the dataset is **highly imbalanced** only 12% of the customers churn. This means that even if the model predicts most customers as "Not Churn," it will still get high accuracy, but it may fail to correctly identify the actual churners.
In the confusion matrix above, although the model achieves **87% accuracy**, it only identifies 25% of actual churners (Recall), which is quite poor. This can lead to significant business losses if churners are not properly identified.
So, metrics like **Precision**, **Recall** and especially **F1-Score** are more suitable in this scenario.

iii)

If the goal is to **minimize customer loss**, then **Recall should be prioritized**.

- **Recall** measures how many actual churners were correctly identified by the model.
- A high Recall ensures that we are catching most of the customers who are likely to leave.
- Even if we mistakenly identify some non-churners as churners (lower Precision), it's still better because those customers can still receive the retention offer — the key is **not to miss real churners**.

In business terms, it is often more acceptable to offer retention benefits to a few customers who wouldn't have churned than to **miss actual churners** who end up leaving.

b)

K-Fold Cross Validation is a technique used to evaluate the performance of a machine learning model by dividing the dataset into *K* equal parts (folds). The model is trained on *K–1* folds and tested on the remaining one. This process is repeated *K* times, each time using a different fold as the test set. Finally, the average performance across all folds is computed.

How it works:

- Shuffle the dataset randomly.
- Split the dataset into *K* folds (e.g., 5 or 10).
- **For each fold:**
  Use the fold as the test set.
  Use the remaining *K–1* folds as the training set.
- Train the model and record the performance metric (e.g., accuracy, precision).
- Average the results from all *K* iterations.

Advantages:

- **More reliable evaluation**: Gives a better estimate of model performance than a single train-test split.
- **Efficient use of data**: Every observation is used for both training and testing.
- **Reduces variance**: Helps identify whether a model is overfitting or underfitting by testing it on multiple data splits.

Disadvantages:

- **Computationally expensive**: Training the model *K* times can be time-consuming, especially for large datasets or complex models.
- **Not ideal for time-series data**: In time-dependent problems, random shuffling breaks temporal relationships.
- **May not reflect real-world deployment**: Especially when data is imbalanced or not IID (independent and identically distributed).

c)

Overfitting occurs when a machine learning model learns the training data too well — including its noise and outliers — and fails to generalize to unseen data. In other words, the model performs very well on the training set but poorly on new, unseen data.
In the context of **Decision Trees**, overfitting happens when the tree becomes too deep or complex, capturing unnecessary patterns or noise.

Measures to prevent overfitting in Decision Trees:

1. **Pruning**:
   - **Pre-pruning (early stopping)**: Stop the tree from growing once it reaches a maximum depth or minimum number of samples.

	• **Post-pruning**: Allow the tree to grow fully, then remove branches that do not improve performance on validation data. 2. **Set a Maximum Depth**: Limit how deep the tree can go using max_depth parameter. 3. **Minimum Samples for Split/Leaf**: Use min_samples_split and min_samples_leaf to prevent splitting nodes that have too few samples. 4. **Use Cross-Validation**: Helps determine the best tree complexity that generalizes well. 5. **Feature Selection**: Remove irrelevant features that can introduce noise. **6.** **Ensemble Methods**: Use models like **Random Forests** or **Gradient Boosting** which average across many trees, reducing variance and overfitting.
Q3.	a) Explain the steps of the K-Means Clustering algorithm. How do you choose the right number of clusters and write its limitations? b) You are given a bank dataset containing the following features: • DD (Demand Drafts) • Withdrawals • Deposits • Branch Area (in sq. ft.) • Average Daily Walk-ins c) Perform K-Means clustering on the given dataset by selecting relevant features, appropriately preprocessing the data (including handling missing values and scaling), determining the optimal number of clusters using the Elbow Method, and then applying the K-Means clustering algorithm based on the identified number of clusters. Interpret the resulting clusters from question (b) in terms of bank operations.
Ans:	a) **K-Means Clustering** is an unsupervised machine learning algorithm used to partition data into *K* distinct, non-overlapping groups (clusters), where each data point belongs to the cluster with the nearest mean. **Steps of the K-Means Algorithm:** 1. **Choose the number of clusters (K)**: Decide in advance how many clusters you want to find. 2. **Initialize centroids**: Randomly select *K* points from the dataset as initial cluster centers (centroids). 3. **Assign points to the nearest centroid**: For each data point, compute its distance to each centroid and assign it to the closest one. 4. **Update centroids**: For each cluster, recalculate the centroid as the mean of all the points assigned to it. 5. **Repeat steps 3 and 4**: Continue reassigning points and updating centroids until the assignments no longer change or a maximum number of iterations is reached.

How to Choose the Right Number of Clusters (K):

1. **Elbow Method**:
   - Plot the Within-Cluster Sum of Squares (WCSS) against different values of K.
   - The "elbow point" where the curve bends indicates the optimal number of clusters.
2. **Silhouette Score**:
   - Measures how similar an object is to its own cluster compared to other clusters.
   - Higher silhouette scores indicate better clustering.
3. **Gap Statistic**:
   - Compares the WCSS for the actual data with that of a random distribution.
   - A larger gap value suggests a better clustering.

Limitations of K-Means:

1. **Need to pre-define K**: The algorithm requires the number of clusters as input, which may not always be obvious.
2. **Sensitive to Initialization**: Poor initial centroids can lead to suboptimal clustering.
3. **Assumes spherical clusters**: K-Means assumes that clusters are convex and isotropic, which may not hold in real data.
4. **Sensitive to outliers**: Outliers can significantly distort the centroids and clustering.
5. **Not suitable for non-linear data**: It may not perform well when clusters have complex shapes or densities.

b) Written in .pynb file, please find the attachment for this answer

c)

Based on the K-Means clustering performed in **Q2. b)** using the features:

- **Demand Drafts (DD)**
- **Withdrawals**
- **Deposits**
- **Branch Area (in sq. ft.)**
- **Average Daily Walk-ins**,

the Elbow Method suggested that the optimal number of clusters is **3**.

After applying K-Means with **K = 3**, the dataset was divided as follows:

- **Cluster 0**: 176 branches
- **Cluster 1**: 181 branches
- **Cluster 2**: 158 branches

The clustering helps the bank **segment branches based on operational profiles**, which can support:

- Targeted staffing or resourcing.
- Infrastructure planning (e.g., cash handling).
- Marketing strategies based on customer footfall and transaction types.

Interpretation of Clusters in Terms of Bank Operations:
Below are the **average characteristics** of each cluster (after inverse scaling):

Feature	Cluster 0	Cluster 1	Cluster 2
Demand Drafts	160	256	286
Withdrawals	144	197	105
Deposits	80	88	75
Branch Area (sqft)	3026	2682	3122
Avg Daily Walk-ins	520	675	598

Cluster 0 – Medium Activity Branches

- Moderate in all features: demand drafts, walk-ins, and withdrawals.
- Likely represents branches with balanced but average customer traffic and operations .

Cluster 1 – High Transaction Volume Branches

- Highest number of walk-ins, withdrawals, and demand drafts.
- Likely large, urban or high-demand branches handling significant transaction volume.

Cluster 2 – High DD, Low Withdrawals Branches

- Highest in demand drafts but lowest in withdrawals.
- Possibly corporate-focused or semi-specialized branches with large area, high walk-ins, and less cash withdrawal activity.

Answers

Section:

Q4. a) Consider the following dataset showing patient record (each row represents the symptoms and diagnosis observed in one patient).

i. For the rule Fever → Flu, calculate Support, Confidence and lift.

	ii. Interpret the rule Fever → Flu based on the support, confidence, and lift values. Is this rule useful for early diagnosis? Justify. b) You are provided with an online retail dataset that contains transaction-level data, including InvoiceNo, Description, Quantity, and InvoiceDate. Using the Apriori algorithm, perform a market basket analysis on this dataset under the following constraints: • Minimum Support: 0.2 • Minimum Confidence: 0.8 • Minimum Lift: 6.0 c) Based on the results from question(b), provide a clear interpretation of the discovered association rules and explain how they can be used to derive actionable insights in a retail business scenario.
Ans:	a) **i. Calculate Support, Confidence, and Lift for the rule: Fever → Flu** **Step 1: Count of Total Transactions** There are **5** patients → Total transactions = **5** **Step 2: Support** Support is the proportion of transactions that contain **both Fever and Flu**. • Patients with **Fever and Flu**: ✓ Patient 1 ✓ Patient 2 ✓ Patient 5 → Total = **3** • **Support (Fever → Flu)** = 3 / 5 = **0.60** **Step 3: Confidence** Confidence is the proportion of transactions with **Fever** that also contain **Flu**. • Patients with **Fever**: ✓ Patient 1 ✓ Patient 2 ✓ Patient 3

- ✓ Patient 5

  → Total = **4**

- **Confidence (Fever → Flu)** = 3 / 4 = **0.75**

Step 4: Lift

Lift measures how much more likely Flu is when Fever occurs, compared to Flu occurring randomly.

- Patients with **Flu**:
  - ✓ Patient 1
  - ✓ Patient 2
  - ✓ Patient 4
  - ✓ Patient 5

    → Total = **4**
- **Support(Flu)** = 4 / 5 = **0.80**
- **Lift(Fever → Flu)** = Confidence / Support(Flu) = 0.75 / 0.80 = **0.9375**

Final Metrics:

- **Support:** 0.60
- **Confidence:** 0.75
- **Lift:** 0.9375

ii. Interpretation of the Rule Fever → Flu

- **Support = 0.60**:

  This means 60% of all patient records include both **Fever and Flu**. That's a relatively high support and indicates that this pattern is fairly common in the data.
- **Confidence = 0.75**:

  Among patients with **Fever**, 75% also had **Flu**. This suggests that **Fever is a strong indicator of Flu**, but not definitive.
- **Lift = 0.9375**:

  Since lift is **< 1**, it means **Fever and Flu co-occur slightly less often than expected**

by chance. This weakens the usefulness of the rule — it might be correlated, but not strongly enough to be a dependable rule.

Is this rule useful for early diagnosis?

Not significantly. While the **support and confidence are reasonably high**, the **lift value below 1** indicates that **Fever does not increase the likelihood of Flu** above the baseline rate. Thus, **Fever alone is not a strong predictor of Flu** - it's likely a **common symptom**, but **not uniquely associated with Flu**.

b) Written in .pynb file, please find the attachment for this answer

c)

Based on the Apriori algorithm results obtained in **question (b)**, we applied the following constraints:

- **Minimum Support:** 0.2
- **Minimum Confidence:** 0.8
- **Minimum Lift:** 6.0

However, the algorithm **did not return any association rules** that satisfied all these conditions.

Interpretation:

This outcome indicates that **no pair or group of items were frequently purchased together** with high enough strength (support, confidence, and lift) to be considered a **strong rule**. Although no rules met the strict constraints in this analysis, adjusting parameters or using a richer dataset could help uncover valuable patterns. Market basket analysis remains a powerful tool for understanding customer purchasing behaviour in retail.

This could be due to several reasons:

- The dataset may be **too small** to reveal strong patterns.
- There could be **too much diversity in product purchases**, with few items co-occurring in more than 20% of transactions.

- The **thresholds set are strict**, especially the lift value of 6.0, which means we're only considering item pairs that are 6 times more likely to be bought together than at random.

Business Insight:

While no strong associations were found under the given constraints, this result itself is **valuable**. It suggests that:

- The business may need to **analyze a larger dataset** over a longer time to identify reliable purchase patterns.
- Alternatively, **lowering thresholds** (e.g., support = 0.1, lift = 3.0) might reveal meaningful relationships.

If meaningful rules were discovered, they could be used to:

- **Design product bundles** (e.g., "Customers who bought Item A also buy Item B").
- Improve **cross-selling strategies** (recommend related products).
- **Optimize store layout or website suggestions** to place commonly purchased items closer together.